The Frequency Domain

Week 6 — Session 2: Periodic noise, phase & the transform family

Practical Image Processing

Lecture 6.2 ▪ 60–70 min

Recap of Session 1

- Every image is a **sum of 2D sinusoids**; the DFT re-describes it on frequency axes.
- We read spectra: centre = DC, directions = perpendicular bright lines, off-centre peaks = repeating structure.
- We filtered by **masking**: low/high-pass, ideal vs smooth (ringing).

Today

The pay-off: kill **periodic noise** with a notch, understand **phase**, meet the **FFT**, **DCT** and **wavelets**, and see **aliasing** + lighting in frequency.

Roadmap

Periodic noise & notch filtering

Amplitude vs phase

The FFT & the transform family

Aliasing & lighting

In-class exercise

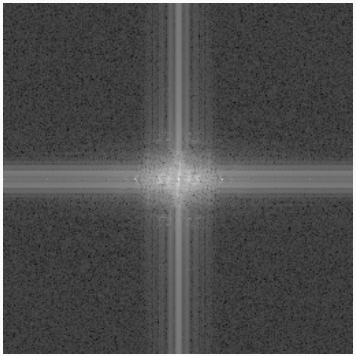
When to use it & wrap-up

Periodic noise & notch filtering

Periodic noise: the killer app



striped noise



spectrum: 2 bright peaks



after notch: clean

Key idea

Repeating stripes are a nightmare in the spatial domain — but two dots in frequency. Zero the dots (a **notch** filter), invert, done.

Where do the two peaks come from?

Math

A pure stripe pattern is a cosine: $n(x, y) = A\cos\big(2\pi(u_0 x + v_0 y)\big)$.

Euler: $\cos\theta = \frac{1}{2}(e^{j\theta} + e^{-j\theta})$.

- So a single cosine is exactly **two** complex exponentials, at $+(u_0, v_0)$ and $-(u_0, v_0)$.
- In the spectrum that is two bright **deltas**, symmetric about the centre.
- Higher stripe frequency $\rightarrow$ peaks sit farther from the centre.

Key idea

Every periodic pattern lands on a small number of isolated points. That is exactly why it is easy to remove here and impossible to remove pixel-by-pixel.

Notch filtering

- A **notch** filter zeroes small regions of the spectrum (the noise peaks) and keeps the rest.
- Find the peaks (they're symmetric about the centre), mask them, inverse-transform.
- Perfect for scan lines, halftone patterns, sensor interference — real document artifacts.

✓ Good practice

For DocScan: if a scanned form has a repeating background pattern, a notch filter removes it cleanly.

Worked example: locating a peak

Math

Image 512×512, centred spectrum. A stripe repeats every **8 px** horizontally. Cycles across the width: $512/8 = 64$.

- The peak sits 64 px left *and* right of centre on the horizontal axis: columns $256 \pm 64 = 192$ and 320, row 256.
- A finer stripe (every 4 px) $\rightarrow$ 128 cycles $\rightarrow$ peaks at ± 128 — farther out.
- Mask a small disc (radius $\sim$ 3–5 px) at *both* points, then invert.

Key idea

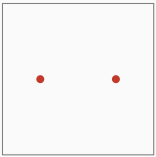
Peak distance from centre = image size $\div$ stripe period. Always notch the symmetric partner too, or you re-inject the stripe.

Notch filter in code

```
F = np.fft.fftshift(np.fft.fft2(noisy))
mag = 20*np.log(np.abs(F) + 1)              # inspect to find peaks
H, W = noisy.shape; cy, cx = H//2, W//2
Y, X = np.ogrid[:H, :W]
notch = np.ones((H, W))
for (py, px) in [(cy, cx+64), (cy, cx-64)]:  # both symmetric peaks
    D = np.sqrt((X-px)**2 + (Y-py)**2)
    notch[D <= 4] = 0                        # zero a small disc
clean = np.abs(np.fft.ifft2(np.fft.ifftshift(F * notch)))
```

`fft2` → zero the peak discs → `ifft2`. Same three-step workflow, targeted mask.

Notch vs band-reject



notch: kill named points



band-reject: kill a ring

- **Notch**: remove a few specific frequencies — best when you can see the peaks.
- **Band-reject**: remove an annulus of frequencies — when noise fills a whole ring (e.g. a rotationally-symmetric texture).

Amplitude vs phase

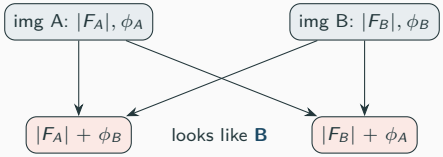
Amplitude vs phase

- The DFT has **magnitude** (how much of each frequency) and **phase** (where it sits).
- Surprising fact: **phase** carries most of the *structure* — swap two images' phases and you swap what you see.
- We visualize magnitude, but don't forget phase exists and matters.

Math

$F(u,v) = |F(u,v)|\, e^{j\phi(u,v)}$ — amplitude $|F|$ and phase ϕ together carry the full image.

The phase-swap thought experiment



Combine A's amplitude with B's phase and you *see* B. Edges and object positions live in the **phase**; the amplitude mostly sets contrast per frequency.

The FFT & the transform family

FFT: why it's fast

- A naive DFT is $O(N^2)$; the **Fast Fourier Transform** is $O(N \log N)$.
- This is what makes frequency-domain filtering practical on real images.
- Numpy/OpenCV give you the FFT; you just prepare the mask and invert.

Math

For a 1024^2 image, FFT is $\sim$50,000$\times$ fewer operations than the naive sum.

Worked example: counting the savings

Math

Treat the image as $N = 1024^2 \approx 10^6$ samples. Naive DFT $\approx N^2$; FFT $\approx N \log_2 N$.

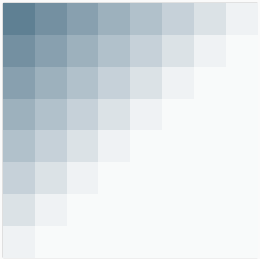
- Naive: $(10^6)^2 = 10^{12}$ operations.
- FFT: $10^6 \times 20 = 2 \times 10^7$ operations.
- Ratio: $10^{12}/(2 \times 10^7) = 5 \times 10^4$ — the $\sim$50,000$\times$ speed-up.

Key idea

The FFT is not a different transform — it is the *same* DFT computed cleverly by reusing shared sub-sums. Without it the frequency domain would be a textbook curiosity.

Other transforms: the DCT (JPEG)

- **DCT** (cosine transform): the heart of JPEG.
- Split the image into 8×8 blocks; transform each; keep the low-frequency coefficients, quantize the rest away.
- Real photos concentrate energy in the top-left of the block $\rightarrow$ big compression.



8×8 DCT block: energy top-left

Note

JPEG's blocking artifacts are exactly these 8×8 blocks becoming visible when you quantize too hard.

Other transforms: wavelets

- **Wavelets**: multi-scale, localized in *both* space and frequency.
- Fourier tells you *which* frequencies exist but not *where*; wavelets keep some spatial location at each scale.
- Great for **denoising** (shrink small coefficients) and compression (JPEG-2000).

Math

Uncertainty trade-off: you cannot be perfectly sharp in both position *and* frequency — each transform picks a point on that curve.

Aliasing & lighting

Aliasing, in the frequency view

- Sampling **replicates** the spectrum on a grid.
- If the spectrum is too wide, the copies **overlap** — high frequencies fold into low ones.
- That fold-over is the moiré / jaggies you saw in Week 1.



spectral copies overlapping

Anti-aliasing = narrow the spectrum first

- Blur (**low-pass**) before downsampling so the spectrum is narrow enough that copies don't overlap.
- This is the frequency-domain reason for the "blur then resize" rule from Week 1.
- The cutoff should be the **Nyquist** frequency of the target resolution.

✓ Good practice

When shrinking a meter photo, always apply a Gaussian pre-filter — otherwise the digit edges alias into false patterns that wreck OCR.

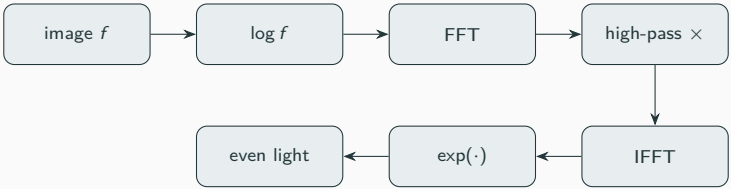
Homomorphic filtering

- Uneven lighting is low-frequency; reflectance (the content) is higher-frequency.
- Take the `log`, then high-pass slightly to suppress illumination and boost detail.
- Useful for a document photographed under a gradient of light — flattens the shading.

✓ Good practice

Another DocScan trick: homomorphic filtering can even out lighting before thresholding.

The homomorphic pipeline



The `log` turns the *product* illumination$\times$reflectance into a *sum*, so a linear high-pass can separate them. Then `exp` undoes the log.

In-class exercise

In-class exercise (10 min)

Design a notch

A 256×256 scan has diagonal stripes repeating every ~ 16 px along the diagonal.

1. How many cycles fit across the image along that diagonal?
2. Roughly where (relative to centre) do the two peaks appear?
3. What radius would you give each notch disc, and why not larger?
4. Why must you zero both symmetric peaks?

Debrief

- (1) Diagonal length $\approx 256\sqrt{2} \approx 362$ px; $362/16 \approx 23$ cycles.
- (2) Two peaks on the *diagonal* through the centre, ~ 23 units out each way (perpendicular to the stripes' direction).
- (3) A small radius (3–5 px): large enough to cover the blob, small enough to keep nearby real content.
- (4) The spectrum is conjugate-symmetric; leaving the partner re-creates a (weaker) real cosine and the stripe returns.

Key idea

Notch design is geometry: period $\rightarrow$ distance, direction $\rightarrow$ angle, always in symmetric pairs.

When to use it & wrap-up

When to use frequency vs spatial

- **Frequency**: periodic noise, very large filters, analyzing texture/orientation.
- **Spatial**: local operations, edges, morphology, anything position-dependent.
- Most day-to-day work is spatial; frequency is the right tool for specific jobs.

Key idea

Don't reach for the FFT by default — but when you see a repeating pattern, it's exactly the tool.

Frequency thinking → deep learning

- CNNs learn spatial kernels, but the frequency view explains what they capture (edges = high-freq).
- Some modern architectures operate partly in frequency for efficiency.
- Understanding frequency deepens your intuition for what any filter — hand-made or learned — does.

What you should now be able to do

- Remove periodic noise with a notch filter — locate peaks from the stripe period.
- Explain amplitude vs phase, and why phase holds the structure.
- Reason about the FFT's speed and place it among DCT / wavelets.
- See aliasing and uneven lighting as frequency-domain phenomena.

Pitfalls recap

✗ Pitfall

- Notching only one of the symmetric peaks — the stripe comes back.
- Over-large notch discs — you delete real detail with the noise.
- Forgetting the `log` in homomorphic filtering — illumination stays multiplicative.
- Downsampling without a pre-blur — aliasing folds high frequencies into your image.

This week's deliverable

Due before Week 7 (see `project/`)

1. Compute and display the magnitude spectrum of 3 real images; interpret each.
2. Implement low-pass and high-pass by masking; compare to spatial blur/edges.
3. Add periodic (stripe) noise; locate the peaks; remove it with a notch filter.
4. Show the notch peak coordinates you used and how you derived them from the stripe period.
5. Note: when is frequency-domain the right tool for a DocScan problem, and when not?

An image is a sum of waves.

Some problems that are impossible in space are trivial in frequency.